

Step 4 — A Real Client That Connects and Uses All Three — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (what a real agent's MCP integration has to handle that a demo script doesn't). Read Basic first even if you already know `asyncio` — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The pieces, and where each one lives](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The pieces, and where each one lives

Basic Version

`client.py` is a completely separate program from `server.py` — it doesn't import anything from it. It launches `server.py` as a subprocess (the same way the Inspector has been doing this whole time, just now it's your own code doing it) and talks to it over the MCP protocol.

Everything in the `mcp` SDK's client side is `async` — every call to the server (`list_tools` , `call_tool` , and so on) has to be awaited, inside an `async def` function, run with `asyncio.run(...)` .

Things to use: - `from mcp import ClientSession, StdioServerParameters` - `from mcp.client.stdio import stdio_client` - `StdioServerParameters(command="python", args=["server.py"])` — describes how to launch the server. - `async with stdio_client(server_params) as (read, write):` then `async with ClientSession(read, write) as session:` — two nested `async with` blocks; both need to stay open for as long as you're talking to the server. - `await session.initialize()` — the protocol handshake; must happen before anything else.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Once `session.initialize()` has run, four kinds of calls matter for this step:

```
tools = await session.list_tools()
resources = await session.list_resources()
prompts = await session.list_prompts()

tool_result = await session.call_tool("add_note", {"text": "..."})
resource_result = await session.read_resource("notes://all")
prompt_result = await session.get_prompt("summarize_notes", {"start_date": "..."},
```

Each of `list_tools()`, `list_resources()`, and `list_prompts()` returns an object with a `.tools`, `.resources`, or `.prompts` attribute — a list of objects, each with a `.name` (and, for tools, a `.description`). Print `[t.name for t in tools.tools]` rather than the raw object, so the output is actually readable.

`call_tool(name, arguments)` takes the tool's name as a string and its arguments as a plain dict — matching exactly what you'd type into the Inspector's form for that tool. The result comes back wrapped in a `CallToolResult`; the actual text is inside its `.content` list.

Write the whole script as one `async def main():` function, ending with `asyncio.run(main())` at the bottom — same shape as the throwaway check script from Step 1's Advanced hint, just doing more now.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

A demo script that calls each thing once and prints the result proves the loop works, but a real agent's MCP client (what Doc06's "why MCP matters for the multi-agent systems this curriculum builds" section was describing) has to handle a few things this simple version can skip:

What if the server process fails to start at all — a typo in `server.py`, a missing dependency? `stdio_client(...)` will raise inside the `async with`, and an uncaught exception there gives a Python traceback that doesn't clearly say "the server subprocess never came up." Wrapping the connection attempt in a `try / except` and printing a clear message ("Could not start or connect to the notes server: ...") is the same "fail loudly, with a clear message" habit from every earlier project in this curriculum, now applied to a subprocess launch instead of an API call.

What if a tool call itself fails — `call_tool("add_note", {"text": ""})`, deliberately, is worth trying here too. `CallToolResult` carries an `isError` flag; check it (`if result.isError:`) rather than assuming every tool call succeeded, print the result, and move on. A real agent has to branch on this exact flag to decide whether to retry, try something else, or tell the user it failed — it's the client-side mirror of Step 2's server-side validation.

What if the server offers more tools/resources/prompts later than it does today? This client should never hardcode "there are exactly 2 tools" anywhere in its logic — only ever loop over whatever `list_tools()` actually returns. That's the entire point of MCP's discovery model: a client written today should keep working, unmodified, if Step 3's server later grows a third Tool.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate connect and call things successfully, assuming everything goes right. Advanced adds the failure handling and the "never hardcode what the server offers" discipline that separates a one-off demo script from something that resembles a real agent's MCP integration.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
launch server.py as a subprocess, open a session, initialize it

print the list of tools, resources, and prompts the server offers

call add_note with some text, print the result
call search_notes with a matching query, print the result

read the notes://all resource, print it

get the summarize_notes prompt with two dates, print it
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# client.py
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            print("Tools:", [t.name for t in tools.tools])

            resources = await session.list_resources()
            print("Resources:", [r.uri for r in resources.resources])

            prompts = await session.list_prompts()
            print("Prompts:", [p.name for p in prompts.prompts])

            add_result = await session.call_tool("add_note", {"text": "Buy milk at the store"})
            print("add_note result:", add_result)

            search_result = await session.call_tool("search_notes", {"query": "milk"})
            print("search_notes result:", search_result)

            resource_result = await session.read_resource("notes://all")
            print("notes://all:", resource_result)

            prompt_result = await session.get_prompt(
                "summarize_notes", {"start_date": "2026-09-01", "end_date": "2026-09-01"}
            )
            print("summarize_notes prompt:", prompt_result)

asyncio.run(main())
```

Run this against your Step 3 server (`python client.py` , with `server.py` in the same folder) and confirm all 6 print statements produce sane output before looking at Advanced.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the missing piece from Hint 1 — a connection failure handled cleanly, and a tool result's `isError` flag actually checked, instead of assumed away. Fill in the two `...` sections yourself:

```
async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    try:
        async with stdio_client(server_params) as (read, write):
            async with ClientSession(read, write) as session:
                await session.initialize()
                # ... discovery + calls from Intermediate go here ...

                bad_result = await session.call_tool("add_note", {"text": "  "})
                if bad_result.isError:
                    print("add_note correctly rejected empty text:", bad_result)
                else:
                    # your turn: what should happen here? think about
                    # what it would mean if this branch ran instead
                    ...
    except Exception as e:
        print(f"Could not start or connect to the notes server: {e}")
```

Fill in your own handling for the `else` branch, run the whole script once more with a deliberately broken `server.py` (rename it temporarily) to see the `except` branch fire with a clear message, then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate prove the full loop works when everything goes right. Advanced proves it fails *usefully* when something doesn't — a broken server, or a tool call that was supposed to fail and did — which is the actual bar a real agent's MCP client has to clear, not just a demo script's happy path.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)